

Curso de Python para Economistas

Universidad Nacional de La Plata

Octubre, 2022

Temario de la clase

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Repaso de la última clase

En la clase 4 vimos:

- Continuamos trabajando con tablas de datos estructurados (clase 3)
- Profundización en Pandas
- Visualizaciones con seaborn
- Utilización de widgets
- Expresiones Regulares

Objetivos de la clase

- La clase de esta semana presenta distintas herramientas útiles para acceder a diversas fuentes de datos.
- Se busca exponer los paquetes más utilizados para hacer web scraping, pdf scraping, conexión a APIs, etc.
- Además, se conversará acerca de las dificultades que pueden aparecer en cada proyecto en particular y que estrategias pueden servir para sortearlas.
- Se espera que estas herramientas sean útiles para quienes esten pensando en sus proyectos de investigación/tesis.

Liberías principales

Algunas de las librerías más utilizadas para los propósitos de esta clase son:

- **Requests:** Permite hacer solicitudes de HTTP
- **BeautifulSoup4:** permite extraer información de paginas web
- **PyPDF2:** simplifica la manipulación de PDFs
- **Tabula:** extrae tablas de PDFs
- **Minecart:** facilita la extraccion de texto, imagenes, colores y formas de PDFs.
- **Sqlite3:** permite conectarse a bases de datos relacionales

Para poder seguir esta clase a la par, con la consola o un Jupyter Notebook, por favor instalen las librerías indicadas en el documento que acompaña esta presentación.

- PDF Scraping
- Web Scraping
- Descargar con un click (csv, excel)
- Conectarnos a una API
- Base de datos relacionales (tipo SQL)

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Descargar con un click

Con el paquete 'requests' podemos enviar solicitudes HTTP/1.1. para obtener el contenido de una url. Es como si estuviéramos accediendo a la url desde el navegador y descargando el contenido del archivo.

```
1 >>> def descarga_base_infectados(csv_url, csv_name):
2     #Obtenemos el contenido del archivo en el url
3     #especificado
4     req = requests.get(csv_url)
5     url_content = req.content
6
7     #Guardamos el contenido descargado en un archivo
8     #con el nombre que elegimos en 'csv_name'
9     csv_file = open(csv_name, 'w')
10    csv_file.write(url_content.decode())
11    csv_file.close()
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Web scraping: ¿Qué es?

- 'Web scraping' es la extracción de datos de sitios web.
- Es una herramienta para obtener información de forma programática y reproducible
- Utilizaremos web scraping cuando queramos datos de un sitio web que no tenga API
- Hay que ser cuidadosos que nuestro web scraping no este violando ninguna regla de privacidad o del sitio
- La calidad de los datos que extraigamos va a depender de la calidad del sitio
- El navegador utiliza instrucciones en HTML para mostrar el sitio web.
- Detectamos el patrón en el HTML y definimos las secciones a extraer

Web scraping: ¿Qué es un HTML?

- HTML son las siglas de Hyper Text Markup Language
- HTML es el lenguaje de marcado estándar para crear páginas web
- HTML describe la estructura de una página web
- HTML consta de una serie de elementos que le dicen al navegador cómo mostrar el contenido.
- Los elementos HTML etiquetan partes de contenido para indicar "esto es un encabezado", "esto es un párrafo", "esto es un enlace", etc.

Web scraping

<code><!DOCTYPE></code>	Sirve para definir el tipo de documento
<code><html></html></code>	Define un documento HTML
<code><head></head></code>	Dentro se escribe información sobre el documento
<code><title></title></code>	Para escribir el título del documento. Se escriben dentro de <code><head></code>
<code><body></body></code>	Justo después de <code></head></code> . Es el contenedor del documento
<code><h1></h1></code> , <code><h2></h2></code> hasta <code><h6></h6></code>	Para crear títulos de párrafos, de más importante <code><h1></code> a menos <code><h6></code>
<code><p></p></code>	Los párrafos de texto de nuestro documento
<code>
</code>	Un salto de línea simple
<code><hr></code>	Salto de línea que indica un cambio de temática entre párrafos
<code><a></code>	Podremos escribir enlaces a otras páginas

fuelle: <https://websgalicia.es/blog/etiquetas-html/>

Pueden consultar otras etiquetas (tags) en **CheetSheets de HTML**

Web scraping

DataTables Editor CloudTables

[Manual](#) [Download](#) [Login / Register](#)

Using DataTables

Examples

Basic initialisation

[Advanced initialisation](#)

[Styling](#)

[Data sources](#)

[API](#)

[Ajax](#)

[Server-side](#)

[Plug-ins](#)

Manual

Reference

Extensions

Plug-ins

[More Help](#)

[Blog](#)

[Forums](#)

[Support](#)

[FAQs](#)

[Get DataTables](#)

[Download](#)

[Purchase](#)

Zero configuration

DataTables has most features enabled by default, so all you need to do to use it with your own tables is to call the construction function: `\$.fn.DataTable\(\)`.

Searching, ordering and paging goodness will be immediately added to the table, as shown in this example.

Show entries

Search:

Name	Position	Office	Age	Start date	Salary
Airi Satou	Accountant	Tokyo	33	2008/11/28	\$162,700
Angelica Ramos	Chief Executive Officer (CEO)	London	47	2009/10/09	\$1,200,000
Ashton Cox	Junior Technical Author	San Francisco	66	2009/01/12	\$86,000
Bradley Greer	Software Engineer	London	41	2012/10/13	\$132,000
Brenden Wagner	Software Engineer	San Francisco	28	2011/06/07	\$206,850
Brielle Williamson	Integration Specialist	New York	61	2012/12/02	\$372,000
Bruno Nash	Software Engineer	London	38	2011/05/03	\$163,500
Caesar Vance	Pre-Sales Support	New York	21	2011/12/12	\$106,450
Cara Stevens	Sales Assistant	New York	46	2011/12/06	\$145,600
Cedric Kelly	Senior Javascript Developer	Edinburgh	22	2012/03/29	\$433,060

NamePositionOfficeAgeStart dateSalary

Showing 1 to 10 of 57 entries

Previous123456Next

Sitio: https://datatables.net/examples/basic_init/zero_configuration.html

Web scraping

```
<!DOCTYPE html>
<html>
  <head>...</head>
  <body class="wide comments example" data-new-gr-c-s-check-loaded="14.983.0">
    <a name="top" id="top">...</a>
    <div class="fw-background">...</div>
    <div class="fw-container">
      <div class="fw-header">...</div>
      <div class="fw-nav">...</div>
      <div class="fw-body">
        <div class="content">
          <h1 class="page_title">Zero configuration</h1>
          <div class="info" style="min-height: 8em;">...</div>
          <div class="demo-html">
            <div id="example_wrapper" class="dataTables_wrapper">
              <div class="dataTables_length" id="example_length">...</div>
              <div id="example_filter" class="dataTables_filter">...</div>
              <table id="example" class="display dataTable" style="width: 100%;" role="grid" aria-describedby="example_info">
                <thead>...</thead>
                <tbody>
                  <tr role="row" class="odd">
                    <td class="sorting_1">Airi Satou</td>
                    <td>Accountant</td>
                    <td>Tokyo</td>
                    <td>33</td>
                    <td>2008/11/28</td>
                    <td>$162,700</td>
                  </tr>
                  <tr role="row" class="even">
                    <td class="sorting_1">Angelica Ramos</td>
                    <td>Chief Executive Officer (CEO)</td>
                    <td>London</td>
                    <td>47</td>
                    <td>2009/10/09</td>
                    <td>$1,200,000</td>
                  </tr>
                  <tr role="row" class="odd">
                    <td class="sorting_1">Ashton Cox</td>
                    <td>Junior Technical Author</td>
                    <td>San Francisco</td>
                    <td>66</td>
                    <td>2009/01/12</td>
                    <td>$86,000</td>
                  </tr>
                </tbody>
              </table>
            </div>
          </div>
        </div>
      </div>
    </body>
  </html>
```


Web scraping

```
1 >>> import requests
2 >>> from bs4 import BeautifulSoup
3 >>> import pandas as pd
```

```
1 >>> url = 'https://datatables.net/examples/basic_init/
    zero_configuration.html'
```

```
1 >>> response = requests.get(url)
2 >>> soup = BeautifulSoup(response.text, 'html.parser')
3 >>> info = soup.find_all('tr')
```

Web scraping

```
1 >>> print(info)
2 [<tr>
3 <th>Name</th>
4 <th>Position</th>
5 <th>Office</th>
6 <th>Age</th>
7 <th>Start date</th>
8 <th>Salary</th>
9 </tr>, <tr>
10 <td>Tiger Nixon</td>
11 <td>System Architect</td>
12 <td>Edinburgh</td>
13 <td>61</td>
14 <td>2011/04/25</td>
15 <td>$320,800</td>
16 </tr>, <tr>
17 <td>Cedric Kelly</td>
```

Web scraping

```
1 >>> df = pd.DataFrame(columns=['name', 'position', 'office',  
2                               'age', 'start', 'salary'])
```

```
1 >>> for i, item in enumerate(info):  
2     ...     if i != 0 and i != len(info)-1:  
3     ...         data_point = item.find_all('td')  
4     ...         row = []  
5     ...         for value in data_point:  
6     ...             row.append(value.text)  
7     ...         df.loc[i-1] = row
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping**
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Utilizaremos el siguiente PDF: **Boletín IFE - ANSES**

Utilizamos el paquete PyPDF2

```
1 >>> import PyPDF2
```

Utilizamos el metodo 'PdfFileReader' y vamos imprimir el contenido leído en cada pagina con un loop y los metodos 'getPage' y 'extractText'

```
1 >>> pdf_file = open('boletin_ife.pdf', 'rb')
2 >>> pdfReader = PyPDF2.PdfFileReader(pdf_file)
3 >>> paginas = pdfReader.numPages
4 >>> for i in range(paginas):
5 ...     print("Contenido de la pagina {}".format(i))
6 ...     pagina = pdfReader.getPage(i)
7 ...     print(pagina.extractText())
```

Para extraer las tablas del PDF vamos a usar el paquete tabula

```
1 >>> import tabula
2 >>> import pandas as pd
```

Utilizamos el metodo 'read_pdf', le vamos a solicitar la tabla de la pagina 7 y que la devuelva como dataframe

```
1 >>> lst_of_df = tabula.read_pdf('boletin_ife.pdf',
    output_format='dataframe', pages=7)
```

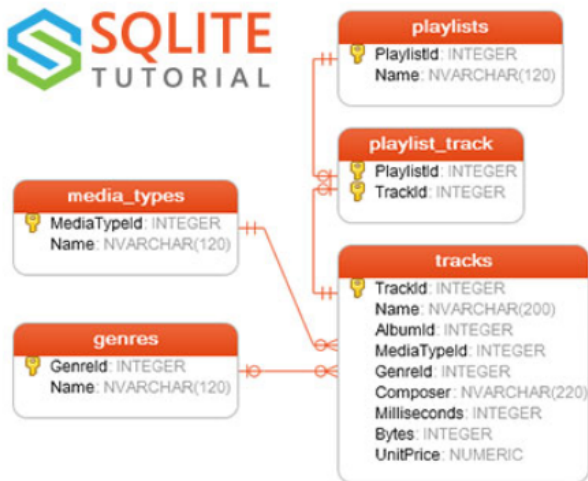
```
1 >>> tabla_p7 = lst_of_df[0]
2 >>> tabla_p7.columns = ['tipo_de_hogar', 'pobreza', '
    indigencia']
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)**
- 6 Conexión a una API

Base de Datos relacionales

Figure: Base de datos relacional



fuelle: <https://www.sqlitetutorial.net/>

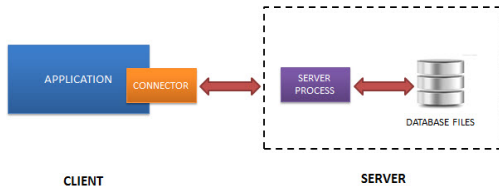
Base de Datos relacionales

Figure: SQLite: No requiere servidor



Self-Contained

Figure: PostgreSQL: Requiere un servidor de BD



fuentes: [sqlite](#) vs [postgresql](#)

Figure: SQLite: Comandos de selección de datos

<code>SELECT *</code>	<i>#Que se quiere seleccionar</i>
<code>FROM hogares_uni</code>	<i>#De que tabla</i>
<code>WHERE total > 10000</code>	<i>#Condición</i>
<code>ORDER BY provincia;</code>	<i>#Variable que ordena</i>

Figure: SQLite: Comandos de edición de tablas

<code>CREATE</code>	<i>#Para crear una tabla</i>
<code>DROP</code>	<i>#Para eliminar una tabla</i>
<code>INSERT</code>	<i>#Para agregar obs. a la tabla</i>

Base de Datos relacionales

Utilizamos el paquete sqlite3

```
1 >>> import sqlite3
```

Iniciamos la conexión a la base de datos con el metodo 'connect' y creamos un objeto cursor para poder comenzar a ejecutar los comandos de SQL:

```
1 >>> conn = sqlite3.connect('IFE.db')  
2 >>> c = conn.cursor()
```

Creo la tabla, inserto una observacion a la tabla

```
1 >>> c.execute('''CREATE TABLE provincias (provincia text,  
2               prov_cod INTEGER PRIMARY KEY)''')  
3 >>> c.execute("INSERT INTO provincias VALUES ('Buenos Aires  
    ', 1)")
```

Insertamos observaciones en bloque a la tabla

```
1 >>> provincias = [('CABA', 2),  
2                   ('Catamarca', 3),  
3                   ('Chaco', 4)]  
4 >>> c.executemany('INSERT INTO provincias VALUES (?,?)',  
                   provincias)
```

Guardamos (commit) los cambios y cerramos la conexión

```
1 >>> conn.commit()  
2 >>> conn.close()
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Conexión a una API

- Las API (application programming interface) son una interfaz informática que define las interacciones entre múltiples intermediarios de software.
- Define los tipos de llamadas o solicitudes que se pueden realizar, cómo hacerlas, los formatos de datos que deben usarse, las convenciones a seguir, etc.
- También puede proporcionar mecanismos de extensión para que los usuarios puedan extender la funcionalidad existente de varias formas y en diferentes grados.
- Las API muchas veces requieren autenticación para garantizar la seguridad de los datos.

Gracias

Esperamos sus dudas en Slack